+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Matplotlib: Personalización avanzada de gráficas

Introducción

Como se indicó en el tema principal de esta semana, Matplotlib es la herramienta de visualización base en Python, gran parte de las herramientas más usadas son construidas sobre Matplotlib debido a su gran versatilidad y personalización. Por tal motivo, es crucial conocer en profundidad los componentes que Matplotlib tiene para la creación de figuras y cómo podemos modificar elementos para crear gráficas según nuestras preferencias y necesidades. En el tema principal usamos el modo indirecto de Matplotlib o *interfaz state-based* para la creación de gráficas. Sin embargo, esto nos limita en la personalización y creación de gráficas. Por lo tanto, en esta guía de profundización usaremos el modo directo o *interfaz object-oriented* el cual nos permite y abre un amplio número de posibilidades y opciones de personalización.

Contenido

A continuación, se abordarán los principales elementos para la creación y personalización de gráficos en Matplotlib.

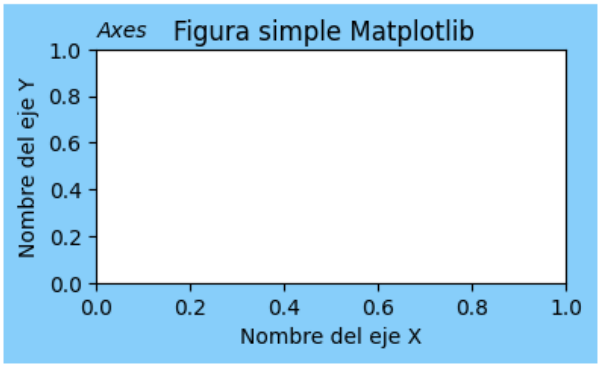
Figuras y lienzos (subplots): Matplotlib gráfica los datos en un elemento llamado Figura (*Figure*) y cada figura contiene uno o más ejes (*Axes*), un área donde los puntos pueden ser especificados en coordenadas *x-y*, *x-y-z* o coordenadas *theta-r* en gráficos polares y además donde coexisten todos los elementos visuales (*Artist*). Iniciamos por la forma más básica para la creación de una figura y algunas opciones de personalización para más adelante agregar complejidad.

```
# Forma más básica para la creación de una figura
# El facecolor color que rodea la figura
fig = plt.figure(figsize=(4, 2), # Tamaño
                 facecolor='lightskyblue')
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# Agregamos un subtítulo para la figura
fig.suptitle('Figura simple Matplotlib')
# Agregamos ejes
ax = fig.add_subplot()
# Agregar un título al lienzo - Posición izquierda
# Tipo de fuente oblicua - Tamaño de fuente media
ax.set_title('Axes',loc='left',
             fontstyle='oblique', fontsize='medium');
# Añadimos nombres a los ejes
ax.set_xlabel('Nombre del eje X');
ax.set_ylabel('Nombre del eje Y');
```

Figura 1: Figura simple en Matplotlib con un único lienzo



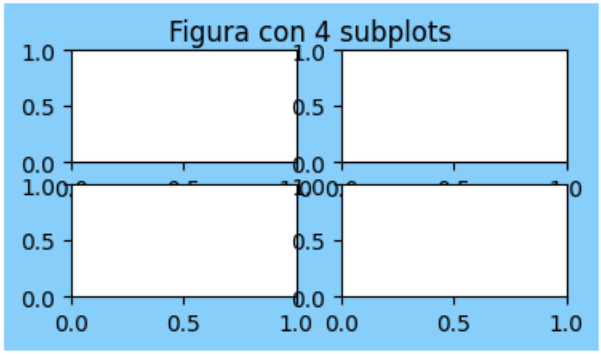
Fuente: Elaboración propia.

La figura resultante muestra como es la forma de crear una figura (especificando el tamaño (ancho, alto)) y añadir algunas opciones de personalización como asignar nombres a los ejes *x* y *y*, tipos de letra y tamaño de la fuente (*xx-small, x-small, small, medium, large, x-large, xx-large, larger, smaller*). La figura anterior solo tiene un eje y fue añadido después de crear la figura. Hay otra alternativa para añadir múltiples ejes desde el principio usando pyplot.subplots.

```
# Añadimos 4 subplots distribuidos en dos columnas y dos filas
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(4,2),
                         facecolor='lightskyblue')
# Agregamos un subtítulo para la figura
fig.suptitle('Figura con 4 subplots');
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 2: Figura en Matplotlib con 4 lienzos



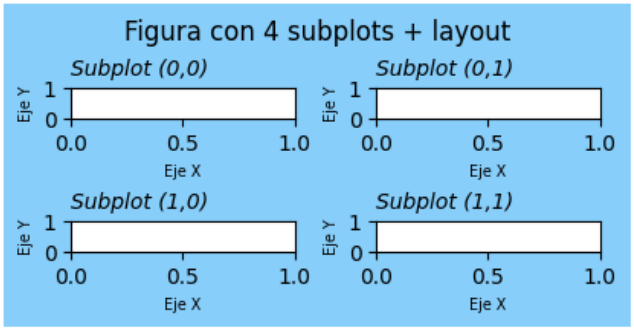
Fuente: Elaboración propia.

El resultado que tenemos es una figura con 4 lienzos para pintar, pyplot.subplots recibe un parámetro que indica cuantos lienzos deseo crear distribuidos en filas y columnas. En nuestro ejemplo, indicamos 2 filas y columnas resultando en 4 lienzos. Además, estos lienzos pueden ser manejados independientemente a través de la variable *axes*. Esta variable es un arreglo en forma de matriz de nfilas x ncolumnas, así que acceder a cada lienzo es cuestión de especificar qué columna y fila deseo usar, miremos un ejemplo:

```
# Importancia de usar el parámetro layout
# Añadimos 4 subplots distribuidos en dos columnas y dos filas
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(4,2),
                         facecolor='lightskyblue',
                         layout='constrained')
# Agregamos un subtítulo para la figura
fig.suptitle('Figura con 4 subplots + layout');
# Agregar un título a cada lienzo - Posición izquierda
# Tipo de fuente oblicua - Tamaño de fuente media
for row in range(2):
  for col in range(2):
    axes[row][col].set_title(f'Subplot ({row},{col})',
                 loc='left', fontstyle='oblique',
fontsize='medium');
    # Añadimos nombres a los ejes
    axes[row][col].set_xlabel('Eje X', fontsize='x-small');
    axes[row][col].set_ylabel('Eje Y', fontsize='x-small');
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 3: Figura en Matplotlib con 4 lienzos sin sobrelapamiento



Fuente: Elaboración propia.

Al final de este ejemplo (en el bucle for), podemos ver cómo se accede a cada lienzo de manera individual, para añadirle el título a cada lienzo. Además, usamos un parámetro llamado *layout,* el cual evita que los elementos dentro de la figura se sobrelapen entre sí, pero se reduce el tamaño del lienzo, por lo que es necesario aumentar el tamaño de la altura de la figura. Ahora, abordaremos otra alternativa para crear lienzos rápidamente y más personalizados, es decir, sin que la distribución de ejes sea simétrica:

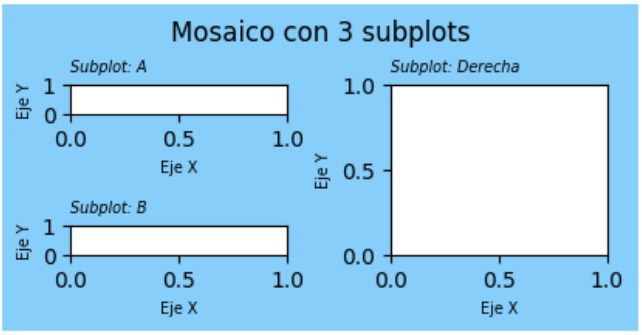
```
# Crear mosaico con 3 lienzos
fig, axes = plt.subplot_mosaic(mosaic=[['A', 'Derecha'],
                                       ['B', 'Derecha']],
                               figsize=(4,2), layout='constrained',
                               facecolor='lightskyblue')
# Agregamos un subtítulo para la figura
fig.suptitle('Mosaico con 3 subplots');
# Los lienzos vienen en un objeto dict de python
for ax_name, ax in axes.items():
  ax.set_title(f'Subplot: {ax_name}',
                loc='left',fontstyle='oblique', fontsize='x-small')
  # Añadimos nombres a los ejes
  ax.set_xlabel('Eje X', fontsize='x-small');
  ax.set_ylabel('Eje Y', fontsize='x-small');
```

Figuras o grillas más complejas pueden ser creadas usando pyplot.subplot_mosaic. En el ejemplo es posible evidenciar cómo se crearon 3

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

lienzos de diferentes tamaños a través de una definición de listas. Algo a notar es que este método no devuelve un arreglo de lienzos sino que devuelve un *dict*.

Figura 4: Mosaico en Matplotlib con 3 lienzos de diferente distribución



Fuente: Elaboración propia.

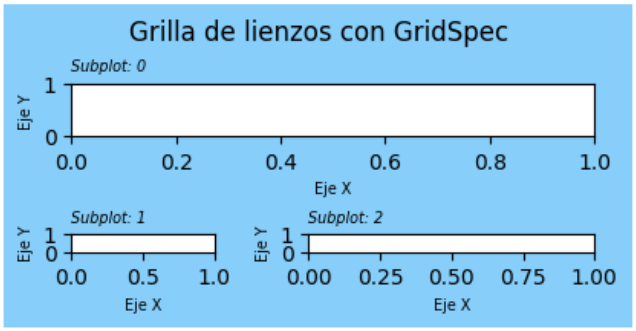
Para entender un poco mejor pyplot.subplot_mosaic podemos revisar la clase que usa por debajo llamada GridSpec. El siguiente ejemplo de más bajo nivel explica cómo crear una grilla de lienzos con más opciones de personalización:

```
# Usar la clase de bajo nivel GridSpec para crear
# una grilla de lienzos similar a lo que hace subplot_mosaic
fig = plt.figure(figsize=(4,2),
                 layout='constrained', facecolor='lightskyblue')
fig.suptitle('Grilla de lienzos con GridSpec')
# Crear el objeto GridSpec
# los ratios de anchura traducen en que la segunda columna es el
doble que la primera mientras que los de altura se traducen en que
# la primera fila es 3 veces mas alta que segunda
gs = fig.add_gridspec(nrows = 2, ncols = 2,
              width_ratios = [1,2], height_ratios = [3,1])
# Añadimos cada lienzo a la grilla para el lienzo ax0 se indica que
debe abarcar toda la primera fila
ax0 = fig.add_subplot(gs[0, :])
ax1 = fig.add_subplot(gs[1, 0])
ax2 = fig.add_subplot(gs[1, 1])
for ax_name, ax in enumerate([ax0, ax1, ax2]):
  ax.set_title(f'Subplot: {ax_name}', loc='left',
                fontstyle='oblique', fontsize='x-small')
  # Añadimos nombres a los ejes
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
ax.set_xlabel('Eje X', fontsize='x-small');
ax.set_ylabel('Eje Y', fontsize='x-small');
```

Figura 5: Grilla con 3 lienzos de distribución diferente usando GridSpec



Fuente: Elaboración propia.

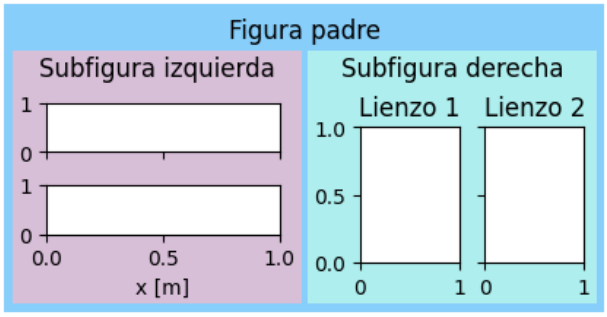
Por otro lado, algunas veces queremos tener un layout anidado a una figura, con dos o más conjuntos de ejes que no compartan la misma grilla. Para esto, es posible usar subfigures que crean figuras virtuales dentro de una figura padre, como se especifica a continuación:

```
# Crear la figura padre o principal
fig = plt.figure(figsize=(4,2), layout='constrained',
                 facecolor='lightskyblue')
fig.suptitle('Figura padre')
# Crear las subfiguras con el n de filas y columnas deseado (es decir
dos subfiguras)
figL, figR = fig.subfigures(nrows = 1, ncols = 2)
# Ajustar el color a la subfigura izquierda
figL.set_facecolor('thistle')
# La subfigura izquierda tiene dos filas
# y una columna y comparte la escala del eje x
axL = figL.subplots(2, 1, sharex=True)
# Colocar el nombre al eje x del segundo lienzo en la subfigura 1
axL[1].set_xlabel('x [m]')
# nombre de la subfigura de la izquierda
figL.suptitle('Subfigura izquierda');
# cambiar el color de la subfigura de la derecha
figR.set_facecolor('paleturquoise')
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
#Añadir dos lienzos a la subfigura de la derecha compartiendo el eje
y
axR = figR.subplots(1, 2, sharey=True)
axR[0].set_title('Lienzo 1'); axR[1].set_title('Lienzo 2')
figR.suptitle('Subfigura derecha');
```

Figura 6: Manejo de subfiguras en Matplolib

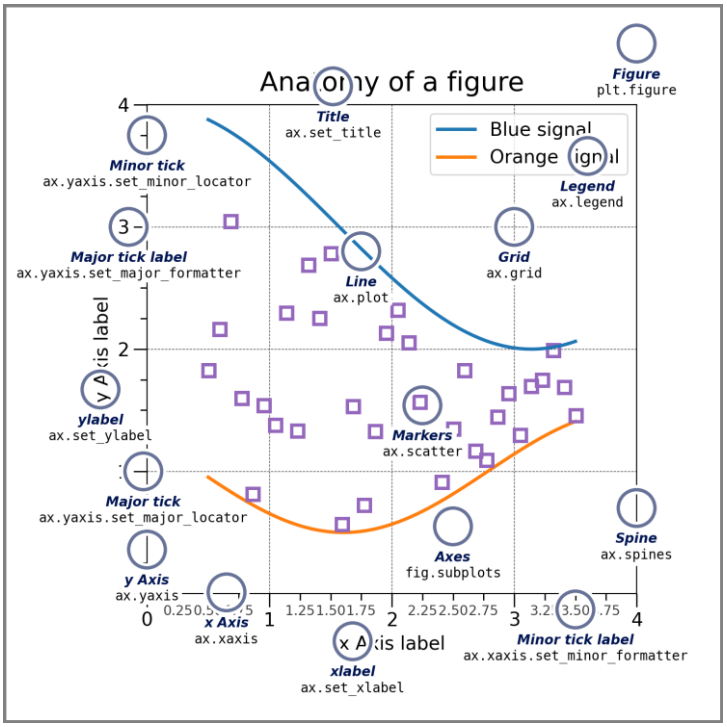


Fuente: Elaboración propia.

De esta manera podemos crear subfiguras. Es importante notar que debemos crear la figura principal y de esta crear las subfiguras para después personalizarlas. En este ejemplo, se usó la propiedad *sharex* y *sharey* que nos permiten compartir la escala de los ejes en los lienzos de cada subfigura. A continuación, la imagen nos permite identificar los elementos de una figura:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 7: Anatomía de una figura Matplotlib



Fuente: Tomado de 'Quick Start guide' por The Matplotlib team, 2023, recuperado de Parts of a figure.

En la siguiente sección abordaremos varias funcionalidades que involucran los elementos presentes en la imagen anterior.

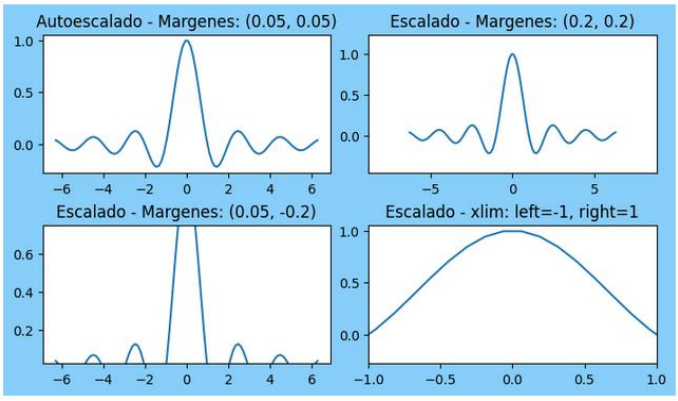
Escalas: Por defecto, siempre que graficamos, Matplotlib fija automáticamente la escala de los ejes basado en los datos, además agrega un margen del 5 % sobre los límites de los datos. Una manera para modificar esto es cambiando el valor de los márgenes del eje (margins) que van desde (-0.5, ∞). Una segunda alternativa es modificando directamente los límites de cada eje. Con esto se desactiva el modo autoescalado. Ejemplo:

```
# Revisemos el escalado automático (5% arriba de los límites)
x = np.linspace(-2 * np.pi, 2*np.pi, 100); y = np.sinc(x)
# Crear la figura
fig, ax = plt.subplots(figsize=(7,4), nrows=2, ncols=2,
                       facecolor='lightskyblue',layout =
'constrained')
ax[0, 0].plot(x,y); # Gráficar
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
ax[0, 0].set_title(f'Autoescalado - Márgenes: {ax[0, 0].margins()}')
# Cambiar los márgenes
ax[0,1].margins(0.2, 0.2)
ax[0,1].plot(x,y);
ax[0,1].set_title(f'Escalado - Márgenes: {ax[0,1].margins()}');
ax[1,0].margins(y=-0.2)
ax[1,0].plot(x,y);
ax[1,0].set_title(f'Escalado - Márgenes: {ax[1,0].margins()}');
# Escalado con limit
ax[1,1].plot(x,y);
ax[1,1].set_xlim(left=-1, right=1);
ax[1,1].set_title(f'Escalado - xlim: left=-1, right=1');
```

Figura 8: Escalado de gráficas en Matplotlib



Fuente: Elaboración propia.

Sobre cada lienzo de la figura es posible modificar la escala de los ejes, podemos especificar sobre el eje *y* o *x* o ambos si se da solo un valor. Entonces tenemos dos opciones una modificando los márgenes u otra modificando directamente el límite del eje con *set_xlim* o *set_ylim*. Otro tema importante a la hora de graficar es el tipo de escala. En Matplotlib por defecto está la escala lineal, pero hay otros tipos como la logarítmica entre otras: *'asinh'*, *'function'*, *'functionlog'*, *'linear'*, *'log'*, *'logit'*, *'symlog*. Ejemplo:

```
# Crear el mosaico
fig, axes = plt.subplot_mosaic([['linear', 'linear-log'],
                                ['log-linear', 'log-log']],
                               layout='constrained',
```

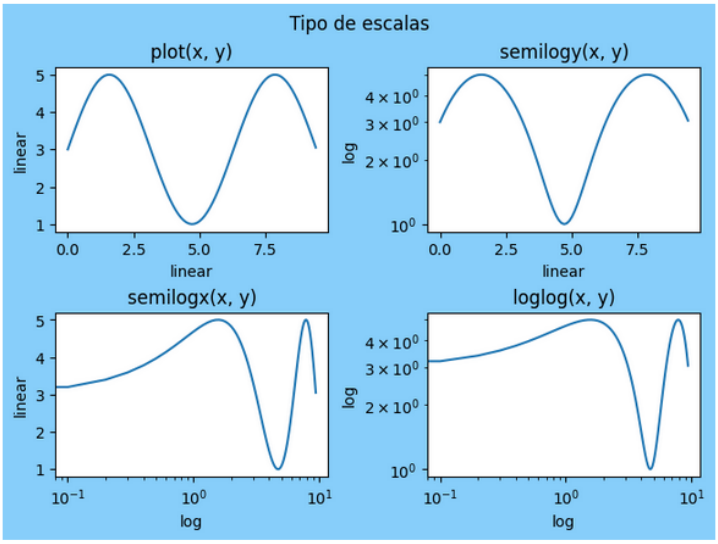
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633


```
                                facecolor='lightskyblue')
fig.suptitle('Tipo de escalas');
# Construir los datos a pintar
x = np.arange(0, 3*np.pi, 0.1)
y = 2 * np.sin(x) + 3
# acceder a los lienzos
ax = axes['linear']
ax.plot(x, y)
ax.set_xlabel('linear')
ax.set_ylabel('linear')
ax.set_title('plot(x, y)');
# cambiar la escala en y
ax = axes['linear-log']
ax.semilogy(x, y)
ax.set_xlabel('linear')
ax.set_ylabel('log')
ax.set_title('semilogy(x, y)');
# cambiar la escala en x
ax = axes['log-linear']
ax.semilogx(x, y)
ax.set_xlabel('log')
ax.set_ylabel('linear')
ax.set_title('semilogx(x, y)');
# cambiar la escala en los dos ejes
ax = axes['log-log']
ax.loglog(x, y)
ax.set_xlabel('log')
ax.set_ylabel('log')
ax.set_title('loglog(x, y)');
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 9: Tipo de escalas en Matplotlib



Fuente: Elaboración propia.

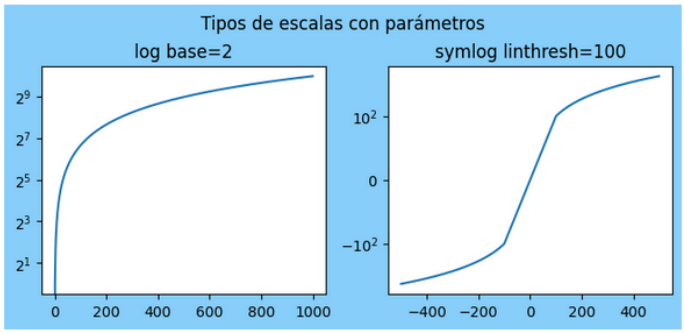
Este es el resultado reemplazando directamente la función plot por el tipo de escala que se desea usar. Sin embargo, otra alternativa, que incluso permite pasar parámetros sería el método *set_xscale* o *set_yscale*. Por ejemplo:

```
# Como pasarle parametros a las escalas,
# por ejemplo, base dos para las logaritmicas
# Crear el mosaico
fig, axes = plt.subplot_mosaic([['log', 'symlog']],
                               layout='constrained', figsize=(6.4,
3),
                               facecolor='lightskyblue')
fig.suptitle('Tipos de escalas con parámetros');
# Crear los datos
x = np.arange(0, 1000)
# Gráficar
ax = axes['log']
ax.plot(x, x)
# cambiar la escala y pasar el parámetro
ax.set_yscale('log', base=2)
ax.set_title('log base=2')
ax = axes['symlog']
ax.plot(x - np.mean(x), x - np.mean(x))
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# cambiar la escala y pasar el parámetro
ax.set_yscale('symlog', linthresh=100)
ax.set_title('symlog linthresh=100');
```

Figura 10: Tipo de escalas con parámetros en Matplotlib



Fuente: Elaboración propia.

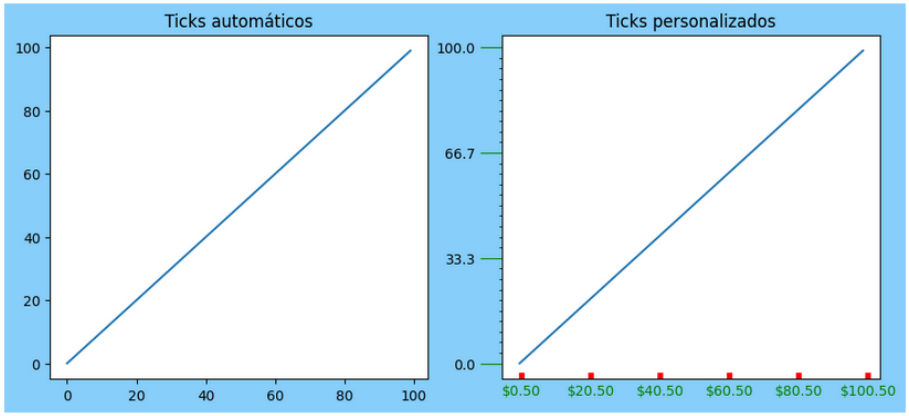
Marcas (Ticks): Los ticks son las marcas en los ejes x y y. Estos dependen de la escala que está siendo usada. Es posible personalizarlos, para tal fin podemos usar métodos de alto nivel como set_xticks o modificando directamente los '*locators*' y "*formatters*". Analizemos un ejemplo usando el método de alto nivel.

```
# Crear la figura con dos lienzos
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(9,4),

layout='constrained',facecolor='lightskyblue')
x = np.arange(100) # Definir los datos
for nn, ax in enumerate(axes):
    # Gráficar
    ax.plot(x, x)
    # modificar los ticks del segundo lienzo
    if nn == 1:
        ax.set_title('Ticks personalizados')
        # Cambiar los ticks del eje y (marcas principales)
        ax.set_yticks(np.arange(0, 100.1, 100/3))
        # modificar los ticks del eje y (marcas más pequeñas)
        # los valores estarán más cercanos
        ax.set_yticks(np.arange(0, 100.1, 100/30), minor=True)
        # cambiar los ticks del eje x
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
        xticks = np.arange(0.50, 101, 20)
        # Si desea agregar labels personalizados a cada tick
        # es posible a través del parámetro labels
        xlabels = [f'\\${x:1.2f}' for x in xticks]
        ax.set_xticks(xticks, labels=xlabels)
        # Opciones para aplicar estilos a los ticks
        ax.tick_params(axis='x', color='red',
length=4,direction='in', width=4, labelcolor='g', grid_color='b')
        ax.tick_params(axis='y', left=True, color='green',
length=16,
                       grid_color='none')
    else:
        ax.set_title('Ticks automáticos')
```

Figura 11: Personalización de ticks en Matplotlib



Fuente: Elaboración propia.

De esta manera podemos modificar varios atributos de los marcadores no solo de escala si no también visuales según nuestras necesidades. Puedes encontrar más opciones de personalización en: tick_params. En la mayoría de casos lo anterior es suficiente para modificar los ticks. Sin embargo, cuando hay necesidades más específicas podemos hacer uso de funciones de más bajo nivel usando "*Locator*" y "*Formatter*" como en el siguiente ejemplo:

```
# Usando locator y formatters
# La función setup aplica personalización a todos los ejemplos
def setup(ax, title):
```

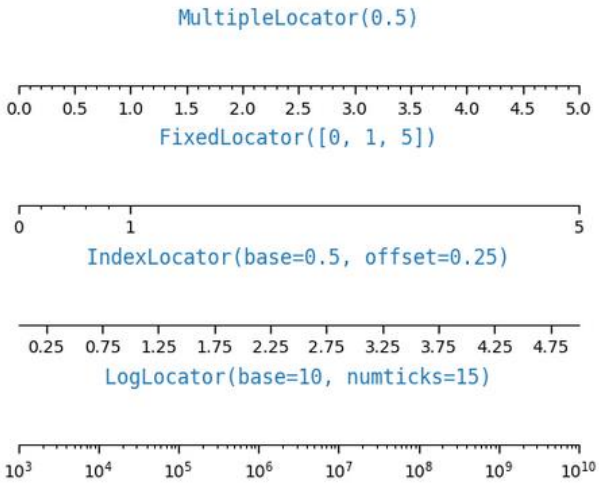
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
    # Spine hace referencia a las líneas del cuadrado de la figura
    # En este ejercicio solo se deja visible la línea de abajo
    ax.spines[['left', 'right', 'top']].set_visible(False)
    # eliminar las marcas del eje y
    ax.yaxis.set_major_locator(ticker.NullLocator())
    # Esta opción nos permite posicionar las marcas en
    # el spin de arriba .. por defecto está en la línea inferior
    ax.xaxis.set_ticks_position('bottom')
    # Para modificar la apariencia de las marcas principales
    # y de las marcas secundarias
    ax.tick_params(which='major', width=1.00, length=5)
    ax.tick_params(which='minor', width=0.75, length=2.5)
    # Fijar los límites para que sea vea mas las diferencias
    ax.set_xlim(0, 5)
    ax.set_ylim(0, 1)
    ax.set_title(title, fontname='Monospace', color='tab:blue')
# Crear la figura
fig, axs = plt.subplots(figsize=(5,4), nrows=4, ncols=1,
                        layout='constrained')
# Multiple Locator
# Coloca las marcas separadas por la distancia
# especificada tanto para los major ticks como para minor ticks
setup(axs[0], title="MultipleLocator(0.5)")
axs[0].xaxis.set_major_locator(ticker.MultipleLocator(0.5))
axs[0].xaxis.set_minor_locator(ticker.MultipleLocator(0.1))
# Fixed Locator
# Esta alternativa permite indicar cuales marcas quiero graficar
setup(axs[1], title="FixedLocator([0, 1, 5])")
axs[1].xaxis.set_major_locator(ticker.FixedLocator([0, 1, 5]))
axs[1].xaxis.set_minor_locator(ticker.FixedLocator(np.linspace(0.2,
0.8, 4)))
# Index Locator - Especificar separado según necesidades
setup(axs[2], title="IndexLocator(base=0.5, offset=0.25)")
axs[2].plot(range(0, 5), [0]*5, color='white')
axs[2].xaxis.set_major_locator(ticker.IndexLocator(base=0.5,
offset=0.25))
# Log Locator
# Graficar en escala logarítmica la base deseada y # de marcas
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
setup(axs[3], title="LogLocator(base=10, numticks=15)")
axs[3].set_xlim(10**3, 10**10)
axs[3].set_xscale('log')
axs[3].xaxis.set_major_locator(ticker.LogLocator(base=10,
numticks=15)); plt.show();
```

Figura 12: Tipos de personalización de Ticks en Matplotlib



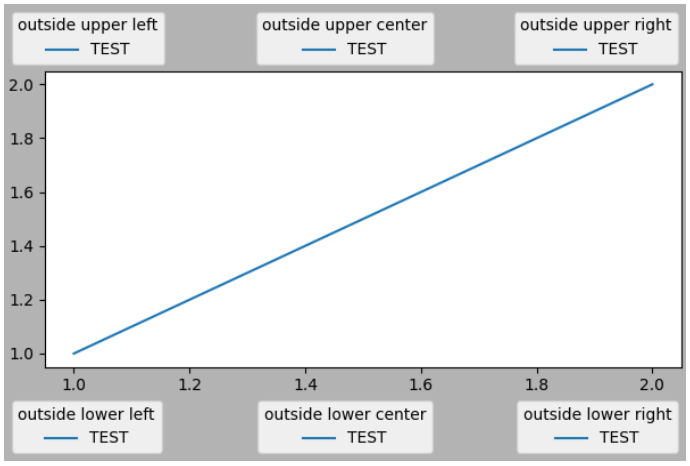
Fuente: Elaboración propia.

Puedes encontrar todas las opciones disponibles: Tick locating and formatting.

Leyendas y Estilos de líneas: Continuamos con aspectos importantes de visualización como lo son las leyendas, estilos de línea y marcadores. Estos elementos son un gran componente visual a la hora de graficar. Con respecto a las leyendas, es importante entender en donde posicionarlas. La siguiente imagen muestra las posibles opciones de posicionar las leyendas en una figura:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 13: Ubicaciones posibles de leyenda en una figura Matplotlib



Fuente: Tomado de 'Axes and Subplots' por The Matplotlib development team, 2023, recuperado de Figure legends.

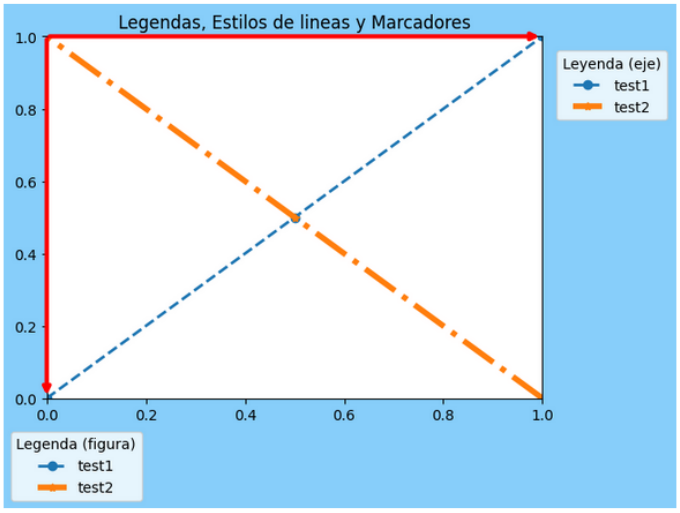
A continuación, se presenta un ejemplo de cómo funcionan estos elementos:

```
# Crear la figura
fig, axes = plt.subplots(facecolor='lightskyblue',
                         layout='constrained')
# Establece los límites del gráfico
axes.set_xlim(0, 1)
axes.set_ylim(0, 1)
axes.set_title('Leyendas, Estilos de líneas y Marcadores')
# Graficar dos líneas de datos con diferentes estilos y marcadores
axes.plot([0, .5, 1], [0, .5, 1],
          label="test1", linestyle='--', linewidth=2, marker='o')
axes.plot([1, 0.5, 0],[0, 0.5, 1], label="test2",
          linestyle='-.', linewidth=4, marker='*')
# colocar el cuadro en una posición específica
# bajo la referencia (x, y, ancho, alto)
# donde x y y pueden ser valores entre 0,1 y
axes.legend(bbox_to_anchor=(1, 1),loc='upper left',
                      borderaxespad=1.00, title='Leyenda (eje)')
# Es posible colocar leyenda a nivel de figura
fig.legend(loc='outside lower left', title='Legenda (figura)');
# Agrega ejes coordenados en la esquina para entender
# el sistema de referencia
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
axes.annotate('',  # No se muestra ningún texto
    (1, 1),  # Punto de inicio del eje x
    (0, 1),  # Punto de finalización del eje x
    arrowprops=dict(arrowstyle='->', color='red', linewidth=3)
)
axes.annotate('',  # No se muestra ningún texto
    (0, 0),  # Punto de inicio del eje y
    (0, 1),  # Punto de finalización del eje y
    arrowprops=dict(arrowstyle='->', color='red', linewidth=3)
)
plt.show()
```

Figura 14: Manejo de leyendas en los lienzos de una figura Matplotlib



Fuente: Elaboración propia.

Lo primero que debemos entender es que es muy fácil cambiar el estilo y los marcadores de línea, solo debemos pasarlo directamente en la función que grafica. Puede encontrar todos los estilos de líneas y marcadores en: Estilos de líneas y Marcadores de líneas. Adicionalmente, como se evidenció es posible cambiar el tamaño y color de las líneas. Por otro lado, está el uso de la leyenda. La manipulación de este elemento puede ser un poco confusa, es por eso que en esta guía se enseña un método sencillo para personalizar la leyenda de cada línea. Hay que saber que para cada elemento graficado en el lienzo es necesario

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

colocarle título a través del parámetro "*label*" y que para que la leyenda se visualice se debe hacer el llamado del método "*.legend*". Aquí viene otro punto importante y es que es posible asignar leyendas tanto a la figura como a los ejes, así como se evidencia en la imagen anterior. Entonces, para la figura ya hay 6 posiciones posibles (ver figura 13). Para los ejes se debe configurar su posición a través del parámetro "*loc*" que es ubicación y la posición (*x,y*) del cuadro de la leyenda con el parámetro "*bbox_to_anchor*". Vale la pena aclarar que este parámetro también recibe valores de altura y anchura pero vamos a ignorar estos valores y nos enfocaremos en la configuración de la ubicación y las coordenadas. Entonces, cuando se selecciona un valor para el parámetro "*loc*" (similar a los valores para la leyenda en figura pero sin la palabra *outside*). Es decir si ingresamos "*lower right*" lo que estamos indicando es que la esquina inferior derecha del cuadro de la leyenda se posicione en la esquina inferior derecha del lienzo (todo esto desde la perspectiva del usuario). Además de entender esto, hay que saber que el eje *x* se mueve de izquierda a derecha de 0 a 1 mientras que el eje *y* se mueve de arriba debajo de 1 a 0 (como indican las flechas rojas). Entonces si estamos ubicados (mentalmente) en la esquina inferior derecha y queremos ubicar nuestra leyenda en la esquina superior derecha habrá que indicar los siguiente valores al parámetro *bbox_to_anchor = (0,1).* Lo anterior indicaría que el vértice inferior derecho del cuadro de la leyenda se ubicaría en el vértice del lienzo con coordenadas x=0, y=1 es decir vértice superior izquierdo del lienzo.

Conclusiones

En esta lectura de profundización se aprendieron diferentes elementos para la configuración y personalización de las gráficas en Matplotlib según las necesidades de nuestros proyectos. En este punto el estudiante ya tiene las herramientas necesarias para seleccionar, configurar y personalizar las gráficas de manera avanzada de acuerdo con el objetivo de la gráfica y del proyecto en general. En detalle, el estudiante puede crear figuras con diferente número de lienzos, tamaños y distribuciones. Además, de la selección y graficación en lienzos de manera individual. También se evidenció cómo personalizar la escala de los ejes del lienzo dependiendo del tipo de dato, lo que asegura una mejor visualización y comprensión. Por otro lado, detalles como el tipo de línea, tamaño de línea, fuente y marcadores permiten la identificación y entendimiento de

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

elementos diferentes en el mismo lienzo para efectos de comparación. Finalmente, se abordó cómo posicionar leyendas tanto en figuras como en ejes usando un método práctico. Una parte importante del proceso de aprendizaje es colocar a prueba y en práctica toda la teoría "aprendida" con el objetivo de afianzar tales conocimientos. Por lo anterior, motivamos al estudiante a que afiance los conocimientos aprendidos en el caso práctico de esta semana. Finalmente, en este enlace hay una guía completa de la opción de personalización en Matplotlib: Personalización en Matplotlib.

Referencia

- The Matplotlib team, (2012-2013). Quick Start Guide. Recuperado el 12 diciembre de 2023 de https://matplotlib.org/stable/users/explain/quick_start.html

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Matplotlib & Seaborn: Guía básica

Introducción

Bienvenidos a esta guía básica, revisaremos dos de las bibliotecas de visualización de datos más poderosas y populares en Python; Matplotlib & Seaborn. En esta guía, nos enfocaremos en introducir algunas de las gráficas básicas vistas en el tema principal que pueden crear utilizando estas herramientas, abriendo la puerta a un mundo de posibilidades en el análisis y presentación de datos. Desde gráficos de líneas y barras hasta visualizaciones de dispersión y más. También, aprenderán cómo estas bibliotecas pueden transformar números crudos en representaciones visuales claras y atractivas, además de cuando usar cada una, ventajas y desventajas.

Contenido

1. Revisión general de Matplotlib & Seaborn:

En Python, dos librerías se sitúan como pilares en el mundo de la visualización de datos: Matplotlib, Seaborn. Cada una contribuye de forma única al ecosistema de la visualización, ofreciendo un conjunto de herramientas diversas para los científicos de datos. Existen otras librerías de visualización de datos en Python, como Plotly o Bokeh, que se revisarán en las siguientes clases. Pero, todo se inicia con las proporcionadas anteriormente.

- **Matplotlib**:

En el núcleo de la visualización de datos en Python se encuentra Matplotlib, una biblioteca versátil que permite crear visualizaciones estáticas, animadas e interactivas. La importancia de Matplotlib radica en su flexibilidad y ubicuidad. Como biblioteca básica, proporciona una base sólida para la creación de visualizaciones básicas, al tiempo que ofrece la complejidad necesaria para el trazado avanzado.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Seaborn**:

Seaborn, que se integra perfectamente con Matplotlib, añade una capa de sofisticación estadística a sus visualizaciones. Destaca en la presentación de datos sutiles, ofreciendo una gama de tipos de gráficos adaptados al análisis estadístico. Esta librería simplifica la creación de visualizaciones complejas, proporcionando un puente entre los datos en crudo y la información procesable.

2. Introducción a la graficación en Matplotlib & Seaborn.

En este apartado vamos a adentrarnos en la implementación de algunos de los gráficos explicados anteriormente. El objetivo es brindar las primeras herramientas para la construcción de gráficos básicos en Matplotlib. Tenga en cuenta que, iremos aumentando el nivel para construir gráficas más complejas. Además, incorporaremos Seaborn para evidenciar sus ventajas sobre Matplotlib en la creación de gráficos más estadísticos.

- **Matplotlib**:

Usualmente Matplotlib es algo complejo de aprender para los usuarios nuevos. Sin embargo, abordaremos conocimientos iniciales para que el aprendizaje sea mucho más rápido. Lo primero que debemos conocer es que Matplotlib tiene dos interfaces de uso: Explícita e Implícita. La interfaz explícita es cuando directamente usamos una instancia del objeto `figure.Figure`, el cual se puede ver como un lienzo que contiene ejes (`axes.Axes`) y es donde se visualizan las gráficas. Por otro lado, existe la interfaz implícita, que es la que usaremos en los ejercicios de este documento. Esta interfaz está inspirada en MATLAB y es una interfaz global encapsulada en el módulo `pyplot` el cual grafica en los ejes que estén siendo usados en el momento. Otros puntos importantes a tener en cuenta, es que la apariencia de las gráficas de Matplolib por defecto no son lo suficientemente llamativas. Sin embargo, la librería cuenta con una amplia variedad de configuraciones para mejorar la apariencia.

Gráficos de líneas y barras en Matplotlib: Crear gráficos de líneas y barras utilizando Matplotlib de manera implícita es una tarea sencilla. Primero, consideremos un escenario donde se tiene información de la temperatura de una

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

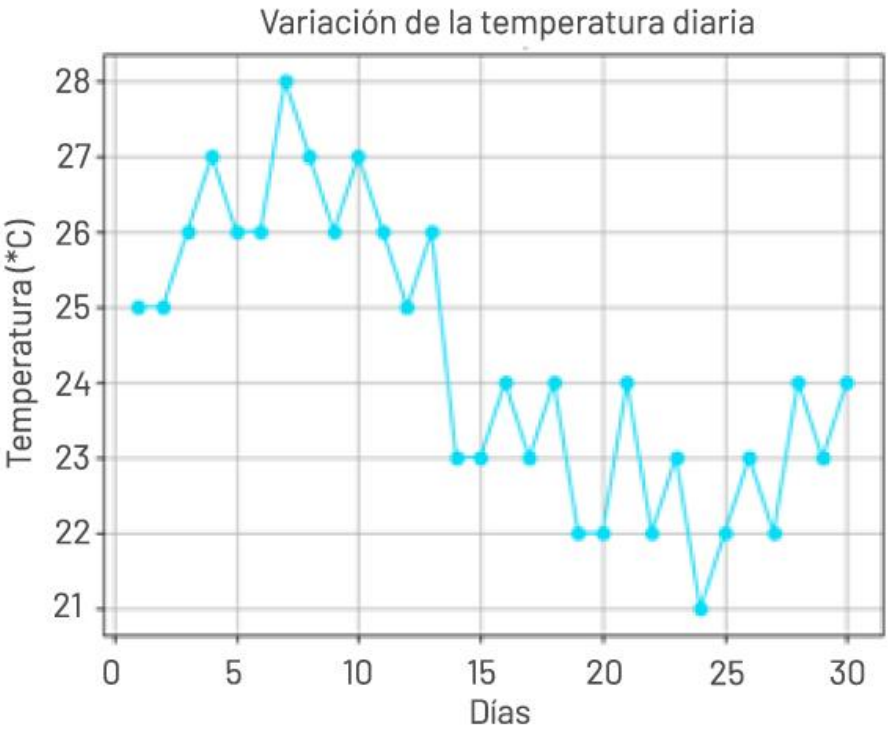
ciudad para todos los días del mes. Como se enseñó podemos usar en este caso un gráfico de línea para visualizar los datos, por lo tanto:

```
import matplotlib.pyplot as plt

# Definir los datos (x = días)
days = [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]
# Crear la temperatura para cada día (y = temperatura)
temperature = [25, 25, 26, 27, 26, 26, 28, 27, 26, 27, 26, 25, 26,
23, 23, 24, 23, 24, 22, 22, 24, 22, 23, 21, 22, 23, 22, 24, 23, 24]
# Crear el gráfico de línea
# Es posible cambia el estilo de la línea y el marcador a usar en
cada punto
plt.plot(days, temperature, marker='o',
         linestyle='-', color='skyblue')
# Agregar título y etiquetado de ejes
plt.xlabel('Días')
plt.ylabel('Temperatura (°C)')
plt.title('Variación de la temperatura diaria')
# Mostrar la grilla de líneas y la gráfica
plt.grid(True)
plt.show()
```

Es importante evidenciar como se importa el módulo pyplot del cual podemos construir la gráfica de líneas directamente. Además, se usaron propiedades de personalización como estilo de línea y marcador de cada dato. Puede revisar los siguientes links para conocer los tipos de líneas y marcadores disponibles además de los colores (Estilos de línea, Marcadores, Colores). Las opciones de personalización dependen del tipo de gráfico, sin embargo, hay opciones globales que modifican toda la apariencia de los gráficos. La gráfica resultante:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 10: Gráfico de línea en Matplotlib



Fuente: Elaboración propia.

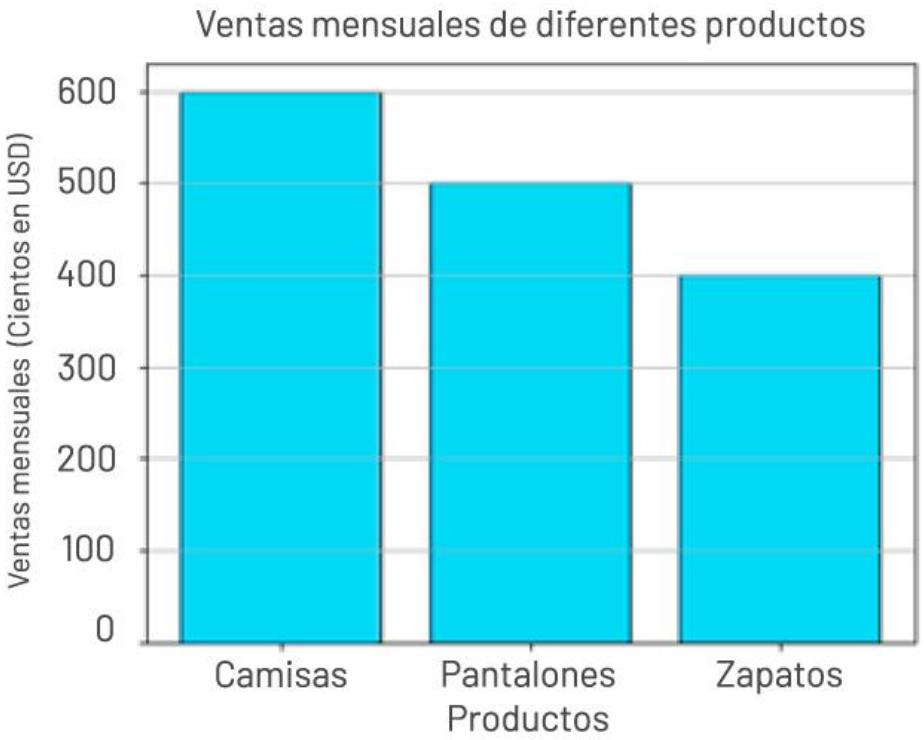
Consideremos ahora otro escenario hipotético en el que tenemos datos de ventas mensuales de diferentes productos. Para lo cual, un gráfico de barras es ideal. El siguiente fragmento de código ilustra cómo construir un gráfico de barras:

```
import matplotlib.pyplot as plt # Método implicito

# Datos de ejemplo (Parejas de datos para cada barra)
products = ['Camisas', 'Pantalones', 'Zapatos']
sales = [600, 500, 400]

# Creación del gráfico de barras y características visuales
plt.bar(x=products, height=sales,color='skyblue',edgecolor='black')
plt.xlabel('Productos') # Etiqueta eje x
plt.ylabel('Ventas mensuales (Cientos en USD)') # Etiqueta eje y
plt.title('Ventas mensuales para diferentes productos') # Titulo
plt.grid(axis='y') # Activar lineas horizontales
plt.show() # Mostrar la gráfica
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

El código anterior genera la siguiente figura de barras:

Figura 11: Gráfico de barras en Matplotlib



Fuente: Elaboración propia.

Gran parte de los tipos de gráficas básicas disponibles en Matplotlib siguen la estructura de las gráficas anteriores (Ver tipos de gráficas). Sin embargo, a medida que debemos hacer gráficas más complejas, el código necesario para construirlas empieza a incrementar, y es aquí cuando el uso de Seaborn es una gran ayuda, ya que simplifica la construcción de muchas gráficas. Considere el siguiente ejemplo de una gráfica de barras apiladas en Matplotlib.

```
import matplotlib.pyplot as plt

# Datos sobre la cantidad de productos vendidos por trimestre
products = ['Laptop Dell', 'Laptop ASUS', 'Laptop Apple', 'Laptop
Lenovo']
quarter1_sales = [150, 200, 100, 120]
quarter2_sales = [180, 220, 120, 150]

# Para Crear las barras agrupadas es necesario indicar la posición
# en el eje x, se le suma el ancho de la barra a las posiciones
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

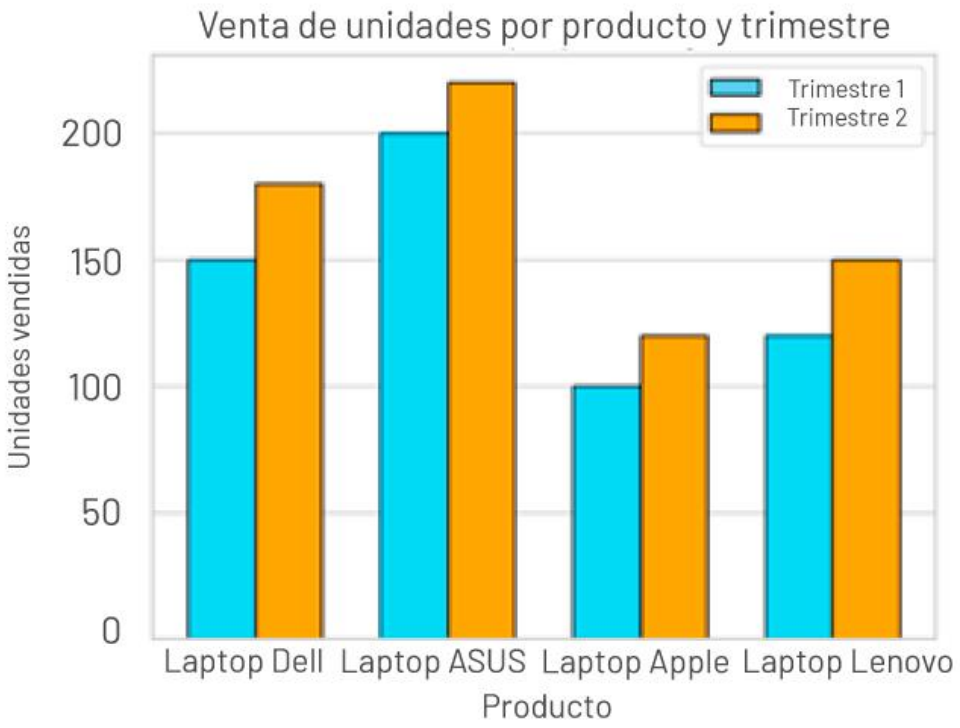
+34 691 225 633


```
# del primer grupo
bar_width = 0.35
r1 = [0, 1, 2, 3]
r2 = [0.35, 1.35, 2.35, 3.35]

# Es decir se crean dos graficos de barras pero matplotlib
# los une en un solo gráfico
plt.bar(r1, quarter1_sales, color='skyblue',
        width=bar_width, edgecolor='black', label='Trimestre 1')

plt.bar(r2, quarter2_sales, color='orange',
        width=bar_width, edgecolor='black', label='Trimestre 2')

# Añadir título y etiquetado de ejes
plt.xlabel('Producto')
plt.ylabel('Unidades vendidas')
plt.title('Venta de unidades por producto y trimestre')

# Hay que posicionar los nombres de cada producto en la mitad
# de las dos barras para cada grupo
plt.xticks([r + bar_width/2 for r in range(len(products))],
products)
# Dibujar la leyenda para cada trimestre
plt.legend()

# Mostrar las lineas horizontales de la grilla y la gráfica
plt.grid(axis='x')
plt.show()
```

Es evidente notar que la cantidad de código para crear una gráfica de barras sencilla y una anidada cambia en gran proporción. Esto es debido a que Matplotlib se enfoca en ser una librería genérica que permita desarrollar y personalizar cualquier tipo de gráfico. Sin embargo, precisamente esto resulta ser una desventaja en algunas ocasiones.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 12: Gráfico de barras agrupadas en Matplotlib



Fuente: Elaboración propia.

Ahora que hemos comprendido cosas básicas de Matplotlib podemos abordar el uso de Seaborn para optimizar y agilizar el tiempo en la creación de gráficas con requerimientos complejos y funcionalidades más estadísticas.

- **Seaborn**:

Seaborn es una librería para hacer gráficos estadísticos en Python. Se basa en matplotlib y se integra estrechamente con las estructuras de datos de Pandas. Sus funciones de trazado realizan internamente el mapeo semántico necesario y la agregación estadística para producir gráficos informativos (Consulte aquí los tipos de gráficas). Veamos algunos ejemplos:

Gráficos para la visualización de distribuciones: Recordemos que si deseamos visualizar la distribución de datos podemos usar histogramas y gráficas de

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

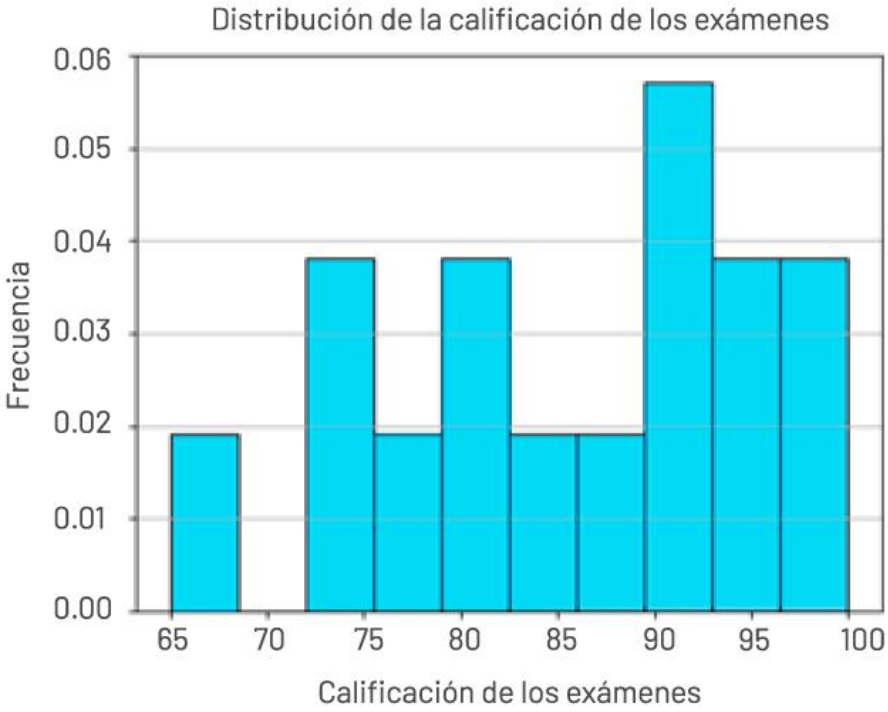
densidad, entre otros. Considere el siguiente escenario donde usaremos Matplotlib y Seaborn para crear un histograma y un gráfico de densidad sobre la información de las calificaciones de 15 estudiantes. Seaborn facilita la construcción de gráficos orientados al análisis estadístico, en este caso las gráficas de densidad.

```
import matplotlib.pyplot as plt

# Datos del puntaje de los exámenes para 15 estudiantes
exam_scores = [65, 75, 80, 85, 90, 95, 87, 76, 92, 95, 98, 100, 74,
81, 90]
# Crear el histograma con matplotlib
# Bins significa el número de grupos que usará para agrupar los
datos
plt.hist(x=exam_scores, bins=10, color='skyblue',edgecolor='black')
plt.xlabel('Calificación de los exámenes')
plt.ylabel('Frecuencia')
plt.title('Distribución de la calificación de los exámenes')
plt.grid(axis='y') # Activar líneas horizontales
plt.show()
```

En los histogramas hay un parámetro que es importante y animamos a explorarlo siempre. Dicho parámetro son los Bins en los que se van a agrupar los datos para contabilizar las frecuencias. Dependiendo del número de Bins escogido, nuestra gráfica puede presentar variaciones visuales. Por lo que se recomienda probar con diferentes Bins para encontrar el que más se adecue a nuestros datos. El código anterior genera la siguiente gráfica:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 13: Histograma en Matplotlib



Fuente: Elaboración propia.

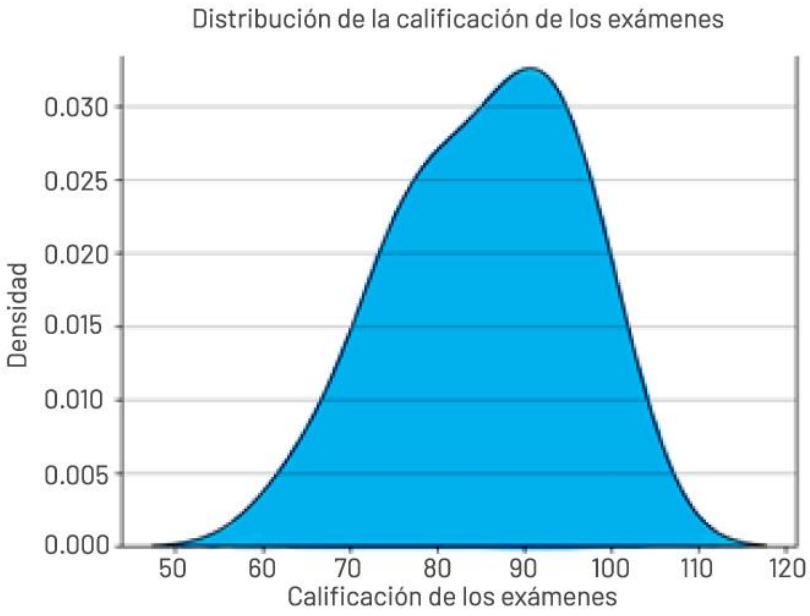
Se observa como Matplotlib agrupa la información de las calificaciones en 10 barras. Ahora, otra manera de visualizar esta distribución, pero esta vez de manera continua, es usando una gráfica de densidad. Veamos como con Seaborn podemos conseguirlo:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Datos del puntaje de los examenes para 15 estudiantes
exam_scores = [65, 75, 80, 85, 90, 95, 87, 76, 92, 95, 98, 100, 74,
81, 90]

# Crear el gráfico KDE (Kernel density estimation) usando Seaborn
sns.kdeplot(exam_scores, fill=True, color='skyblue',
edgecolor='black')
plt.xlabel('Calificación de los exámenes')
plt.ylabel('Densidad')
plt.title('Distribución de la calificación de los exámenes')
plt.grid(axis='y') # Activar lineas horizontales
plt.show()
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Este tipo de gráfica en Matplotlib, hubiese requerido mucho más código. Sin embargo, Seaborn lo simplifica y genera la siguiente gráfica:

Figura 14: Gráfico de Densidad en Seaborn



Fuente: Elaboración propia.

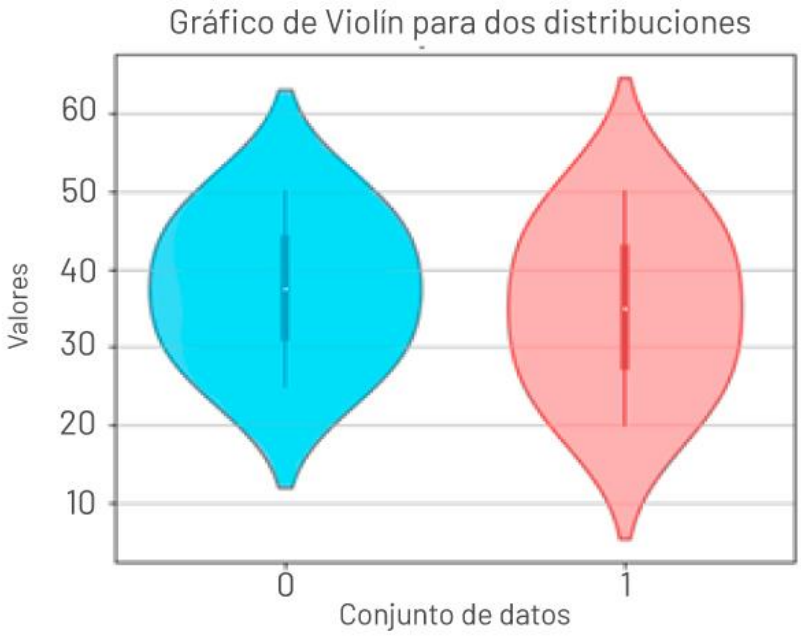
De manera complementaria, Seaborn nos permite crear gráficos para comparar varias distribuciones en una misma gráfica. En este caso es posible usar gráficos de cajas, violín entre otros. El siguiente ejemplo muestra un caso donde tenemos dos conjuntos de datos y queremos comparar sus distribuciones:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Puntos de datos para dos datasets
data_set1 = [25, 30, 35, 40, 45, 50]
data_set2 = [20, 25, 30, 35, 40, 45, 50]

# Crear el gráfico de Violin
sns.violinplot(data=[data_set1, data_set2], palette='pastel')
plt.xlabel('Conjunto de datos')
plt.ylabel('Valores')
plt.title('Gráfico de Violin para dos distribuciones')
plt.grid(axis='y')
plt.show()
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Así de sencillo es crear un gráfico para comprar varias distribuciones en un mismo gráfico. Revisemos la visualización resultante:

Figura 15: Gráfico de Violín en Seaborn



Fuente: Elaboración propia.

Para seguir explorando Seaborn, vamos a usar un gráfico muy útil para explorar relaciones entre variables: Gráfico de Burbuja.

Gráfico de burbuja para representar relaciones de 3 variables: Consideremos un escenario en el que queremos explorar la relación entre dos variables, "X" e "Y", al tiempo que consideramos la magnitud de una tercera variable, "Z". En este hipotético conjunto de datos, "X" e "Y" representan coordenadas en un espacio bidimensional, y "Z" significa una magnitud asociada a cada punto de datos:

- **Eje X:** Representa una variable de interés, quizás las horas dedicadas a actividades relacionadas con el trabajo.
- **Eje Y:** Representa otra variable, como la puntuación de productividad alcanzada durante esas horas de trabajo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Eje Z (Tamaño de las burbujas):** Representa una tercera dimensión, por ejemplo, el valor monetario de los proyectos completados durante esas horas de trabajo.

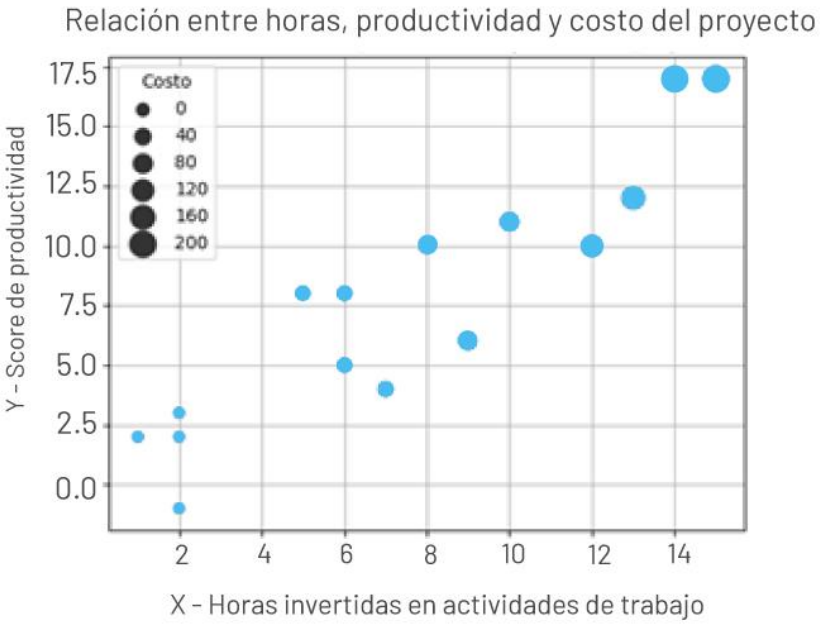
Visualicemos ahora este escenario mediante un gráfico de burbujas, en el que cada punto del gráfico no sólo capta la relación entre "X" e "Y", sino también la magnitud denotada por el tamaño de la burbuja.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Datos de ejemplo para cada variable
X = [2, 7, 2, 6, 14, 6, 10, 5, 13, 1, 12, 8, 9, 2, 15]
Y = [2, 4, 3, 5, 17, 8, 11, 8, 12, 2, 10, 10, 6, -1, 17]
Z = [1, 48, 1, 33, 189, 38, 98, 29, 179, 2, 139, 65, 82, 0, 223]

# Crear el diccionario de datos
data = {'Horas': X, 'Productividad': Y, 'Costo': Z}

# Crear el gráfico de burbuja
# Sizes especifica el rango del tamaño de las burbujas
sns.scatterplot(x='Horas', y='Productividad', size='Costo',
data=data, sizes=(50, 230), legend=True)

# Añadir título y etiquetado de ejes
plt.xlabel('X - Horas invertidas en actividades de trabajo')
plt.ylabel('Y - Score de productividad')
plt.title('Relación entre horas, productividad y costo del
proyecto')

# Mostrar las lineas de la grilla y la gráfica
plt.grid()
plt.show()
```

El parámetro “Size” en este caso siempre será el que no permite visualizar la relación de la tercera variable, con la variable “legend” se activa una leyenda a un lado de la gráfica que muestra la escala para esta variable. Recuerde usar el rango para la variable “size” ya que con respecto a este fija el tamaño de las burbujas. El resultado correspondiente es el siguiente:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 16: Gráfico de Burbuja en Seaborn



Fuente: Elaboración propia.

Así, es como podemos visualizar las relaciones entre 3 variables, si solo quisiéramos revisar dos variables sería un diagrama de dispersión.

Conclusiones

Familiarizarse con las librerías esenciales de visualización como Matplotlib y Seaborn: Recuerde en qué momento usar cada opción para ahorrar tiempo, siempre use Matplotlib para personalizar sus gráficas, una buena presentación añade bastante valor al resultado final. Como Tip importante, Seaborn está construida sobre Matplotlib y está pensado para trabajar con DataFrames de pandas, domine esta herramienta ya que las visualizaciones de Seaborn trabajan con estructuras en formatos “long”.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Referencias

- Moffitt, C. (2017). Practical Business Python, Effectively Using Matplotlib. Recuperado el 20 de noviembre de 2023 de https://pbpython.com/effective-matplotlib.html

- Waskom, M. (n.d.). An Introduction to Seaborn. Recuperado el 20 de noviembre de 2023 de https://seaborn.pydata.org/tutorial/introduction.html

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Fundamentos de la visualización de datos

Introducción

Introducción a la visualización de datos

¡Bienvenido al cautivador mundo de la visualización de datos! En esta sesión, nos embarcaremos en un viaje para desentrañar el poder de la visualización de datos. Prepárese para una exploración de los fundamentos de la visualización de datos, donde descubriremos el arte y la ciencia detrás de convertir datos en historias convincentes.

Introducción a la visualización de datos

Palabras clave

Visualización de datos, Tipos de gráficos, Errores en la visualización de datos, Comunicación visual.

Reto

¿Alguna vez se ha preguntado cómo elegir el tipo de visualización adecuado para sus datos? El reto consiste en saber cuándo emplear gráficos de líneas, de dispersión o de barras. Descubra el lenguaje visual de los datos y domine el arte de elegir el gráfico perfecto para una comunicación eficaz.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Desarrollo

Dominar el arte y la ciencia de la visualización de datos

La visualización de datos es el eje sobre el que gira la ciencia de datos. Es más que crear gráficos; es la alquimia que transforma los datos abstractos en ideas comprensibles. Esta sección profundiza en los fundamentos, asegurándose de que comprenda los principios básicos que sustentan toda visualización de datos eficaz.

1. Papel de la visualización de datos:

La visualización de datos es el medio a través del cual decodificamos el lenguaje de los datos, transformando las complejidades numéricas en historias visuales. Es la llave que abre patrones, tendencias y valores atípicos, ofreciendo una comprensión holística de conjuntos de datos complejos. Imagínese navegar por un conjunto de datos que ilustra indicadores económicos globales: mediante las visualizaciones, puede descubrir correlaciones cargadas de matices, orientar las decisiones políticas y transmitir tendencias económicas complejas con claridad.

2. Tipos de gráficos en visualización de datos:

Exploremos los tipos de gráficos más usuales del variado panorama de los tipos de visualización, cada uno con sus puntos fuertes para transmitir aspectos específicos de los datos.

- **Gráficos para representar cantidades.**

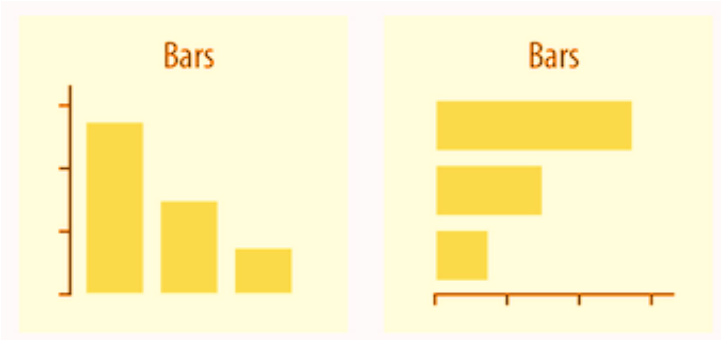
La forma más habitual de visualizar cantidades (es decir, valores numéricos de un conjunto de categorías) es mediante barras, dispuestas vertical u horizontalmente.

Gráficos de barras (Bars plots): Los gráficos de barras son los narradores de los datos categóricos, ya que ofrecen una narrativa clara para las comparaciones. Imagina un conjunto de datos que catalogan las ventas mensuales de varios

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

productos: un gráfico de barras comunica concisamente qué productos dominan el mercado, ayudando en las decisiones de inventario y marketing.

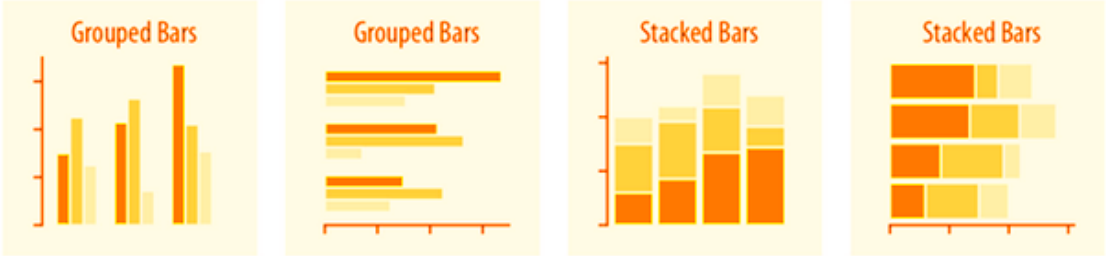
Figura 1: Visualización de cantidades - Gráficos de barras



Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – Amounts'

Además, si hay dos o más conjuntos de categorías para las que queremos mostrar cantidades, podemos agrupar o apilar las barras

Figura 2: Visualización de cantidades - Gráficos de barras apiladas y agrupadas



Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – Amounts'

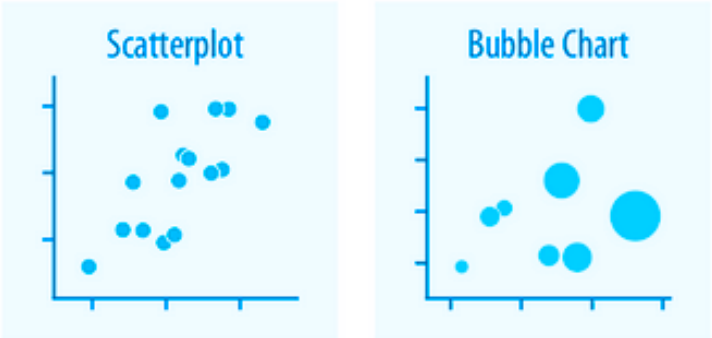
- **Gráficos para representar relaciones**:

Estos gráficos son usados cuando se quiere mostrar una variable cuantitativa en relación con otra. Son importantes, porque ayudan a descubrir fácilmente correlaciones entre los datos permitiendo identificar patrones subyacentes. Los gráficos de dispersión son los más usados para dicho propósito.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Gráficos de dispersión (Scatterplots): Los gráficos de dispersión iluminan las relaciones entre variables, revelando patrones y correlaciones. Un gráfico de dispersión podría mostrar correlaciones entre el tiempo de permanencia en un sitio web y el número de interacciones, lo que permitiría optimizar la experiencia del usuario. Además, si se desea incluir una tercera variable esta puede ser representada como el tamaño de cada punto en la gráfica. Esta variante se suele conocer como gráficos de burbuja (Bubble Chart).

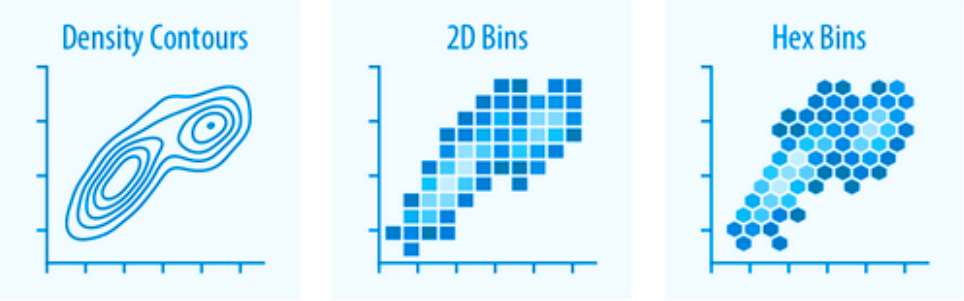
Figura 3: Visualización de relaciones - Gráficas de dispersión



Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – x-y relations

Por otro lado, cuando existe un número grande de puntos, los gráficos de dispersión pasan a ser menos informativos. En este caso, gráficos como líneas de contorno (Density Contours) o gráficos de Bins son una alternativa.

Figura 4: Visualización de relaciones - Gráficas avanzadas de dispersión

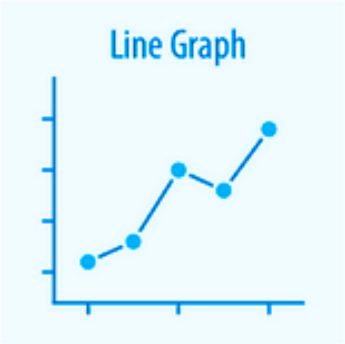


Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – x-y relations

Gráficos de líneas (Line graph): Cuando el eje x representa tiempo, los gráficos de líneas facilitan la descripción temporal, lo que los hace ideales para representar tendencias a lo largo del tiempo. Imagínate un conjunto de datos que recoge las fluctuaciones diarias de temperatura: un gráfico de líneas muestra con elegancia el ascenso y descenso de las temperaturas a lo largo de las estaciones, revelando patrones climáticos a largo plazo.

Figura 5: Visualización de relaciones - Gráficas de línea



Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – x-y relations

- **Gráficos para representar distribuciones**:

Estos gráficos son fundamentales para ilustrar la dispersión, la tendencia central y la forma de un conjunto de datos, ya que proporcionan un resumen visual de cómo se distribuyen los puntos de datos en diferentes valores. Los tipos de gráficos usados comúnmente son los histogramas y los gráficos de densidad.

Histogramas (Histogram): Los histogramas son la lente a través de la cual examinamos las distribuciones de datos, arrojando luz sobre las tendencias centrales. En un conjunto de datos que perfila las calificaciones de los exámenes de los estudiantes, un histograma representa visualmente la frecuencia de las calificaciones, proporcionando información sobre la distribución general del rendimiento.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 6: Visualización de distribuciones - Histogramas

Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – Distributions

Gráficos de densidad (Density Plot): Los gráficos de densidad son una potente herramienta para visualizar la distribución de datos continuos. A diferencia de los histogramas, que agrupan los datos en intervalos, los gráficos de densidad proporcionan una representación suave de la distribución subyacente. Por ejemplo, un gráfico de densidad podría revelar la concentración de puntos de datos en un conjunto de datos sobre salarios, mostrando los picos y las depresiones en la distribución de los ingresos.

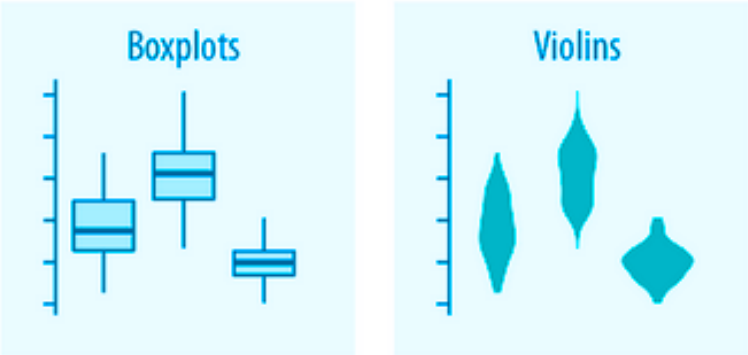
Figura 7: Visualización de distribuciones - Gráficos de densidad

Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – Distributions

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Gráficos de caja y de violín (Boxplot & Violins): Estos tipos de gráficos se usan en los casos cuando se desee observar varias distribuciones al mismo tiempo, principalmente porque el objetivo es evidenciar los cambios existentes entre estas.

Figura 8: Visualización de distribuciones - Gráficos de Cajas & Violín



Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – Distributions

- **Gráficos para representar proporciones**:

Estos gráficos ilustran eficazmente la distribución de las partes en relación con el todo, mostrando de forma clara e intuitiva las relaciones proporcionales entre las distintas categorías o segmentos. Son herramientas valiosas para comunicar los tamaños relativos y las contribuciones de varios elementos dentro de un conjunto de datos.

Gráficos circulares (Pie charts): Los gráficos circulares son ideales para ilustrar proporciones dentro de un conjunto. Piense en un conjunto de datos que represente la participación en el mercado de diferentes productos: un gráfico circular puede transmitir de forma clara la contribución porcentual de cada producto al mercado total, facilitando la comprensión de la distribución global. Otros tipos de gráficos útiles para representar proporciones son las barras apiladas (Stacked Bars) ya vistos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 9: Visualización de proporciones - Gráficos circulares

Fuente: Tomado de 'Fundamentals of Data Visualization' por Santa Clara University team, 2019, recuperado de 'Directory of visualizations – Proportions

Al comprender el propósito y los puntos fuertes de cada tipo de visualización, podrás elegir los medios más eficaces para transmitir tu narrativa basada en datos.

3. Errores comunes en la visualización de datos:

Para crear visualizaciones eficaces no basta con elegir el tipo de gráfico adecuado. Comprender los errores más comunes es crucial para evitar interpretaciones erróneas y mejorar la comunicación.

- **Ejes y escalas engañosos**: Un error frecuente es manipular los ejes o las escalas para exagerar diferencias o similitudes. Asegúrese siempre de que los ejes estén debidamente etiquetados y escalados para representar con precisión los datos.

- **Gráficos abarrotados**: Los gráficos sobrecargados con demasiados puntos de datos pueden confundir en lugar de aclarar. Simplifíquelos cuando sea necesario, centrándonos en las tendencias clave y los valores atípicos.

- **Tipos de gráficos inadecuados**: La selección de tipos de gráficos inadecuados puede tergiversar los datos. Tenga en cuenta la naturaleza de los datos -

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

categóricos, numéricos o temporales- y elija el tipo de gráfico que mejor se adapte a sus características.

- **Uso excesivo del color**: El uso excesivo del color puede distraer y confundir. Utilícelo a propósito para resaltar los elementos clave y mantener la legibilidad.

- **Ignorar a la audiencia**: No tener en cuenta la familiaridad de la audiencia con los datos puede llevar a una simplificación excesiva o a una complejidad abrumadora. Adapte las visualizaciones al nivel de conocimientos de su público.

Conclusiones

Resumen de los fundamentos de la visualización de datos.

Al concluir esta exploración de los fundamentos de la visualización de datos, recuerde los pilares claves:

- **Comprender el papel de la visualización**: Reconozca y esté convencido que la visualización de datos en la ciencia de datos es de suma importancia ya que es la única manera de observar y comprender los datos.

- **Reconocer los tipos de visualizaciones en función de lo que se quiere representar**: Pregunte inicialmente que datos tiene y que quiere presentar o que necesita responder con los datos. En función de esto elija un tipo de gráfico adecuado basado en los estándares aprendidos. Tenga en cuenta que hay algunas variaciones de los tipos de gráficos presentados, indague por esas variaciones y amplíe el abanico de posibilidades al momento de elegir la gráfica correcta.

- **Recordar los errores más comunes para evitarlos**: Tenga muy presente los errores más comunes al momento de graficar, es esencial crear gráficos efectivos para transmitir correctamente el mensaje detrás de los datos. Otro

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

punto poderoso radica en no solo evitar cometer errores sino también incorporar principios de diseño, esto servirá en clases posteriores donde hay que integrar varias gráficas en un solo tablero de visualización.

La capacidad de elegir la visualización adecuada es una habilidad poderosa, similar a la selección de la pincelada perfecta en un lienzo. Esto es sólo el principio; en las próximas semanas le esperan conocimientos prácticos y más profundos usando diferentes herramientas, no solo en Python, y técnicas más avanzadas. Así que mantenga viva la curiosidad y profundice en estos conceptos ya vistos.

Referencias

- Wilke, C. O. (2019). Fundamentals of Data Visualization, Chapter 5: Directory of Visualization. Recuperado el 18 de noviembre de 2023 de https://clauswilke.com/dataviz/directory-of-visualizations.html.

- Schermann, M. (2019). A Reader on Data Visualization, Chapter 1: Introduction. Recuperado el 18 de noviembre de 2023 de https://mschermann.github.io/data_viz_reader/introduction.html

- Healy, K. (2018). Data Visualization: A Practical Introduction, Chapter 1: Look at Data. Recuperado el 20 de noviembre de 2023 de https://socviz.co/lookatdata.html.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

eig Business School | UIDE Powered by Arizona State University

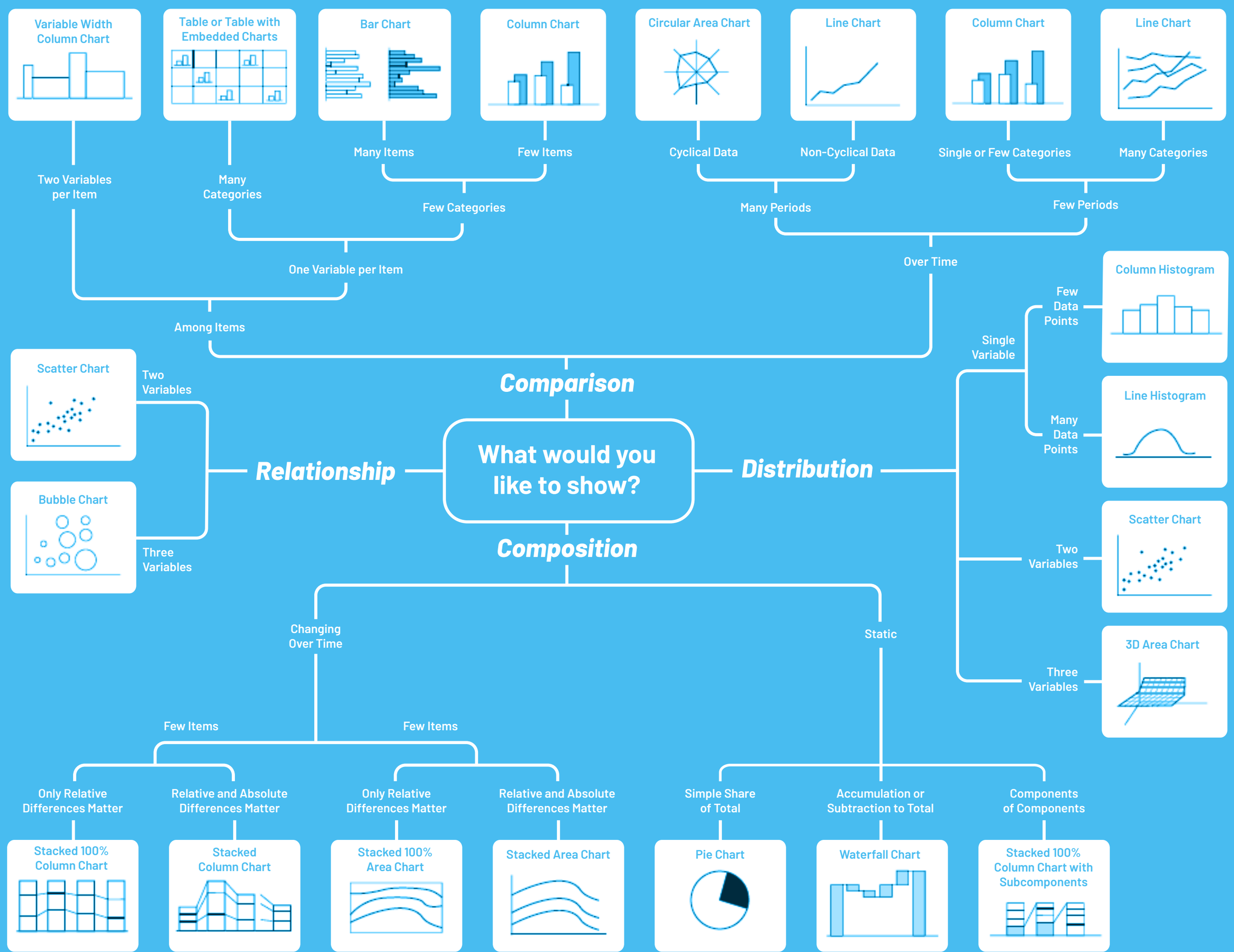



CHART SUGGESTIONS – A THOUGHT-STARTER